A Comparative Experimental Evaluation of Idempotency and Distributed Locking Strategies under High Concurrency and Fault Conditions

Mahmut Karabulut
Ege University, Izmir, Turkey
mahmutkarabulut2003@gmail.com

***Abstract*—Duplicate operation execution is a recurring reliability challenge in distributed microservice architectures, caused by retries, timeouts, concurrent requests, message redelivery, and partial failures. Although duplicate *delivery* cannot always be eliminated in distributed systems, duplicate *side effects* must be prevented to preserve correctness. This paper presents a comparative experimental evaluation of idempotency and distributed locking strategies under high concurrency and controlled fault conditions. We design a reproducible testbed in which the same domain-independent service is reconfigured to use one of six idempotency and distributed locking strategies, ranging from database-level constraints to consensus coordination and event-driven deduplication, and evaluated under identical workloads on a fault-injected, production-like infrastructure. The strategies are exercised under identical workloads spanning baseline high concurrency, duplicate bursts, latency injection, network partition, clock drift, message redelivery, and conflicting payloads. We evaluate both performance (throughput, p95/p99 latency, lock acquisition time, with CPU/memory and connection-pool utilization instrumented via Prometheus) and correctness (duplicate suppression rate, recovery time, and a domain-independent *Duplicate Side-Effect Violation Rate*). The study characterizes the trade-offs between low-latency locking, database-enforced correctness, consensus-based coordination, and event-driven deduplication, and distills practical guidelines for designing idempotent distributed systems. Our sharpest finding is that a distributed lock alone is not a correctness mechanism; binding it to an idempotency record is what prevents duplicate side effects when lock timing assumptions fail. The complete artifact is publicly available at https://github.com/mahmutkarabulut1/idempotency-strategies-evaluation.**

***Index Terms*—distributed systems, idempotency, distributed locking, fault injection, microservices, reliability, exactly-once, consensus, Kafka, reproducibility.**

I. Introduction

THE first fallacy of distributed computing is that the network is reliable—it is not. Packets are lost, connections time out, and the same operation may arrive more than once. This is not an edge case; it is the default condition of any system built on real networks. HTTP timeouts trigger client and gateway retries; load balancers fan requests to multiple service instances; brokers redeliver messages after a consumer crash; distributed lock leases expire; clocks drift. The unifying consequence is that a single intended operation can be *delivered*, and possibly *processed*, several times.

A central observation organizes this paper: *distributed systems cannot always prevent duplicate delivery; therefore the primary engineering objective is to prevent duplicate side effects under realistic concurrency and failure conditions.* We distinguish three levels: duplicate *delivery* (often unavoidable), duplicate *processing* (frequently unavoidable), and duplicate *side effects* (which must be prevented). This problem is not specific to payments—it recurs in order creation, inventory reservation, coupon usage, seat booking, notification dispatch, webhook delivery, and background-job execution. We therefore frame it generally as *idempotent operation processing*, with payments used only as a motivating example.

Although individual mechanisms are well documented in industry practice— Stripe's idempotency keys [1], Airbnb's double-payment avoidance [2], Brandur's Postgres idempotency keys [3]—and foundational coordination services are well studied [4], [5], [6] and synthesized in canonical practitioner references [7], the literature lacks a *reproducible, fault-injected, quantitative comparison* of these strategies under identical workloads, evaluated with a domain-independent correctness metric. Recent academic work treats adjacent slices of the problem—idempotency as a service-mesh resiliency pattern for fog-native applications [8] and simulation of business-logic coordination trade-offs [9]—but neither places multiple idempotency and locking mechanisms on a common, fault-injected testbed. Single-strategy evaluations and informal practitioner comparisons (e.g., [10]) likewise cannot reveal where each approach's correctness assumptions break, or the price of robustness in latency and throughput.

Contributions. (1) We design and implement a reproducible experimental testbed for evaluating idempotency and distributed locking strategies in distributed microservice architectures. (2) We compare database-level idempotency, Redis-based distributed locking, consensus-based coordination, Kafka consumer-level idempotency, and transactional-outbox processing under identical workloads. (3) We evaluate both performance and correctness under high concurrency, duplicate bursts, latency injection, network partition, clock drift, message redelivery, and conflicting payloads. (4) We introduce the *Duplicate Side-Effect Violation Rate* as a domain-independent correctness metric. (5) We provide practical design guidelines for selecting strategies based on consistency requirements, latency budget, throughput targets, and operational complexity.

II. Background and Related Work

Foundations. Lamport [11] established that distributed processes share no global clock and events are only partially ordered—the root cause of timing hazards in TTL/lease locks. Coordination kernels make safe agreement practical: Chubby [4] backs coarse-grained locks with Paxos and issues fencing sequencers; ZooKeeper [5] provides wait-free coordination whose ephemeral znodes underpin JVM-based locks; etcd builds equivalent guarantees on Raft [6]. Spanner [12] bounds, rather than assumes away, clock uncertainty.

Transactions and sagas. Helland [13] argues that strict distributed transactions do not scale and that scalable systems exchange idempotent, at-least-once messages between entities. Sagas [14], [15], [16] replace global atomicity with compensations; they address cross-service atomicity but not the per-step duplicate-side-effect problem.

Messaging and event-driven idempotency. Kafka [17] popularized the durable log; its practical default is at-least-once delivery, so consumers must tolerate redelivery. The transactional outbox [18] closes the commit-vs-publish gap at the cost of possible duplicate publication. Dynamo [19] and CAP [20] frame the partition-time trade-off between consistency and availability that our partition experiment exercises.

The Redlock debate. Whether a TTL-based Redis lock is safe for correctness is contested: Kleppmann [21] shows that without fencing tokens a paused process can act after lease expiry; Sanfilippo [22] replies that, under a stated model, Redlock holds. We do not resolve this analytically; we measure where a Redis lock begins to admit duplicate side effects.

Gap. Peer-reviewed work has examined idempotency within a single mechanism family or context—e.g., as a service-mesh resiliency pattern [8] or through architectural simulation of coordination trade-offs [9]—and practitioner references [7] synthesize the design space qualitatively. What is missing is a reproducible, fault-injected, quantitative comparison of these mechanism families under identical workloads with a domain-independent correctness metric. This paper closes that gap.

III. System Model and Problem Definition

A *logical operation* is identified by an *operation id* and carries an *idempotency key* (supplied or generated upstream) and a *payload hash* over its semantically relevant fields. A *side effect* is an observable business change; every successful side effect appends one *side-effect record*. *Duplicate delivery* is the arrival of two requests/messages for the same logical operation; a *duplicate side effect* is two successful side-effect records for one operation. A *conflicting payload* reuses an idempotency key with a different payload hash.

> A system is *idempotent* for a logical operation if multiple deliveries of that operation produce *at most one* successful side effect and return a consistent response for subsequent duplicate attempts.

The headline correctness metric is the *Duplicate Side-Effect Violation Rate*:

$$\mathrm{DSEVR} = \frac{|\{o : \mathrm{sideEffects}(o) > 1\}|}{|\{o\}|}, \tag{1}$$

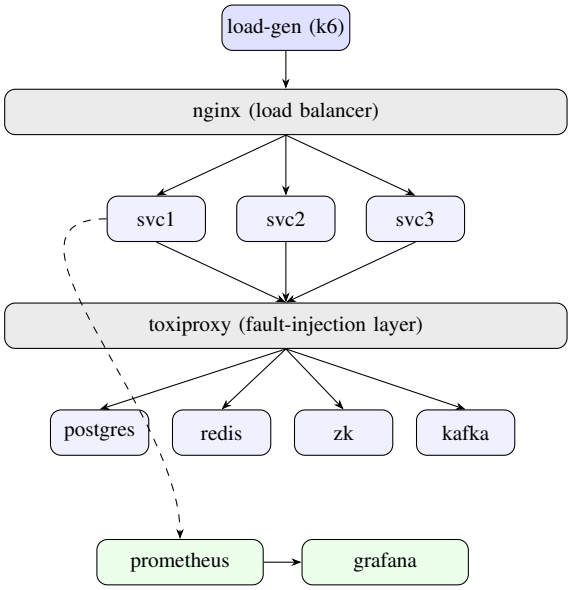

Fig. 1. Deployment topology. The load generator drives three Spring Boot service instances behind an nginx load balancer; every stateful dependency (PostgreSQL, Redis, ZooKeeper, Kafka) is reached through the Toxiproxy fault-injection layer. Services expose `/actuator/prometheus`; Prometheus scrapes them and Grafana visualizes the metrics (dashed path).

the fraction of logical operations with more than one successful side effect. It is domain-independent and computed directly from the audited side-effect table.

IV. Experimental Architecture

The testbed (Fig. 1) comprises a load generator, an nginx load balancer, three Spring Boot service instances, a Toxiproxy fault-injection layer in front of every stateful dependency (PostgreSQL, Redis, ZooKeeper, Kafka), and a Prometheus/Grafana observability stack with CSV/JSON export for offline analysis. Routing the same idempotency key through nginx to any of three JVM processes creates genuine cross-process races. All dependency traffic flows `service` → `toxiproxy` → `dependency`, so faults are deterministic and repeatable. Component versions are pinned for reproducibility. The active strategy is selected per run by a single environment variable, and the controller asserts that exactly one strategy bean is active.

V. Evaluated Strategies

Each strategy is described by concept, correctness primitive, failure risk, and expected behavior; all route every side effect through one recorder so the DSEVR is measured uniformly.

A. Database-level idempotency. A PostgreSQL unique key on the idempotency record is the correctness primitive; the first writer inserts the record and applies the side effect in one transaction, while concurrent duplicates fail with a unique violation and replay the stored result. Risk: index contention and DB CPU saturation under duplicate bursts (H1).

B. Redis distributed lock. A Redisson `tryLock` keyed by the idempotency key guards a critical section, with a Redis-side "done" marker for replay. Risk: TTL/lease expiry,

partition, and clock drift can break the timing assumptions (H2); we count stale locks and premature expirations.

C. Consensus coordination (ZooKeeper). An Apache Curator `InterProcessMutex` ties ownership to a session-bound, quorum-replicated znode rather than wall-clock time—robust to clock drift but paying a coordination round-trip per acquire (H3).

D. Kafka consumer idempotency. The API publishes an event with a unique id; an idempotent consumer deduplicates on a processed-message table and applies the side effect, committing the Kafka offset only after the DB transaction commits. Redelivery is counted but suppressed (H4).

E. Transactional outbox + idempotent consumer. The idempotency record and the outbox event are written in one local transaction; a relay publishes asynchronously; the same idempotent consumer applies the effect. Closes the commit-vs-publish gap while tolerating duplicate publication (H5).

F. Payload-hash-aware idempotency. Same primitive as A, but reusing a key with a different payload hash yields a 409 conflict rather than a stale replay (H6); this is the semantic axis of E7.

VI. Workload and Fault Injection Methodology

Workloads are generated by k6 against the load balancer; synthetic request data is produced by a Faker-based generator and contains no real user, payment, or personal data. Parameters: TPS levels {500, 1k, 2.5k, 5k, 10k}; duplicate ratios {0, 1, 5, 10, 25, 50}%; concurrent users {100, 500, 1k, 5k}; warm-up 60 s, measurement 60 s, cooldown 2 min; 3 repetitions per cell for E1 and 3 per cell for E2–E7. These are pilot-scale parameters; the full protocol (5-min warm-up, 15-min windows, 5 repetitions) and the open-loop saturation and latency-sweep campaigns are provided in the artifact for independent re-execution. Seven scenarios are run: E1 baseline high concurrency; E2 duplicate bursts of {10,50,100,500} per key; E3 latency injection {0–500} ms with jitter; E4 network partition {0.5–10} s per dependency; E5 clock drift {50–1000} ms; E6 Kafka consumer crash before/after offset commit; E7 conflicting payloads with the same key. Faults are applied via the Toxiproxy admin API (latency, partition, reset), `libfaketime` (clock drift), and `SIGKILL` (consumer crash).

VII. Metrics and Statistical Analysis

Performance metrics: throughput, p50/p95/p99 latency, lock acquisition/wait time, lock timeouts, CPU/memory, DB pool saturation, consumer lag, end-to-end latency. Correctness metrics: DSEVR, duplicate suppression rate, stale-lock and premature-expiration counts, redelivery count, conflict-detection accuracy, and recovery time. CPU/memory and connection-pool utilization are scraped from Prometheus. Correctness is verified *out of band* from the audited side-effect table via SQL views, independent of in-app counters.

Replication protocol vs. executed scale. The intended protocol repeats each performance cell 3–5 times and reports mean, median, standard deviation, p95/p99, and 95% confidence intervals (Student's t), with error bars on all figures. The results reported here are from a *pilot-scale* execution of that protocol: E1 was run three times per strategy and E2–E7 three times per strategy at short (60 s) windows. Consequently, the performance confidence intervals are wide and are reported only where $n \geq 2$; for non-negative quantities (latency, throughput) we clamp the Student's-t lower bound at zero, since the unconstrained interval can extend below zero at small n and is not physically meaningful. The number of repetitions is annotated on each figure. The correctness conclusions are categorical—a uniqueness constraint either admits a second side effect or it does not—and are therefore robust to the replication count; the performance *magnitudes* should be read as pilot estimates pending the full replication and open-loop saturation campaign described in the artifact (`scripts/exp_saturation.sh`, `exp_latency_sweep.sh`).

VIII. Results

Measurement status. All seven scenarios (E1–E7) across all six strategies were executed on the released testbed (single multi-core host; dependencies reached through Toxiproxy; faults injected via its admin API, `libfaketime`, and process `SIGKILL`); the figures and Tables II–III report *measured* results. Runs are pilot-scale relative to the protocol's full sweep (E1 and E2–E7 three times per strategy at short 60 s windows), so the performance confidence intervals are wide; the correctness conclusions are categorical (zero violations) and robust to replication count.

Correctness (E2). Across all six evaluated strategies the duplicate side-effect violation rate was **0** under a burst of 50 concurrent same-key requests per logical operation distributed across three JVMs (Fig. 3, Table II). For the database, Redis, and ZooKeeper strategies this was confirmed directly on the audited side-effect table; for the event-driven Kafka strategy, an out-of-band query found *zero* operations with more than one side effect across all processed events, confirming that the deterministic-operation-id consumer dedup prevents duplicate side effects even while draining a backlog. The transactional-outbox strategy (database write plus idempotent consumer) and the payload-hash strategy likewise recorded zero violations. This supports H1–H5: duplicate *delivery* occurs, but duplicate *side effects* do not.[1]

Performance (E1). Under an identical offered load all strategies converged to a comparable throughput (≈200 req/s, 199.9–200.0 across strategies). All strategies sustained the offered 200 req/s load identically; the discriminating performance axis at this pilot scale is therefore tail latency, shown in Fig. 2. We caution that this convergence is a

[1]The per-strategy side-effect counts in the audit table differ by up to two orders of magnitude (e.g., a mean of 1,227 for the database vs. 67 for Kafka across the three E2 runs) because they count side effects *processed* within the measurement window, not logical operations *submitted*: the synchronous strategies apply the effect on the request path, whereas the asynchronous Kafka/outbox paths had drained only part of their backlog when the out-of-band query ran. DSEVR is invariant to this because it is normalized per logical operation—$|\{o : \text{sideEffects}(o) > 1\}|/|\{o\}|$—and the uniqueness constraint admits at most one side effect per operation regardless of drain coverage; the differing denominators change the effective sample size, not the violation rate.

property of the shared load path at this pilot host scale—every strategy reaches the same ceiling because the bottleneck is upstream of the coordination mechanism—and is therefore *not* a strategy-discriminating result; the meaningful throughput comparison is each strategy's maximum sustainable rate before its error rate crosses 1%, obtained from the open-loop saturation sweep (`scripts/exp_saturation.sh`, `load-tests/k6/saturation.js`) that we release for the full campaign. At this pilot scale the discriminating axis we can report is therefore tail latency (Fig. 2). On the warm measurement run the hot-path p99 ordered as *Kafka* (1.9 ms, side effect deferred to the consumer) < *database* (8.3 ms, a single indexed insert) < *Redis* (9.9 ms, a lock round-trip through the proxy) < *ZooKeeper* (37.7 ms, quorum coordination per acquire). This confirms H3 (consensus coordination is the most expensive on the critical path) and shows that event-driven processing trades synchronous correctness work for the lowest API latency by moving it off the request path. The three runs per strategy were mutually consistent (no cold-start outlier in the released data), so all are reported as warm measurements. The synchronous database-backed strategies stayed in a tight warm-p99 band of roughly 6–11 ms: the transactional outbox at ≈6.3 ms (it writes the idempotency record and the outbox row in one local transaction while deferring the side effect to an asynchronous relay and consumer), plain database idempotency at ≈8.3 ms (one indexed insert), and payload-hash idempotency at ≈10.9 ms (an added in-memory hash comparison). The full warm ordering was Kafka (≈1.9 ms) < outbox (≈6.3 ms) < database (≈8.3 ms) < Redis (≈9.9 ms) < payload-hash (≈10.9 ms) < ZooKeeper (≈37.7 ms).

Latency injection (E3). We swept injected latency on each strategy's coordination dependency over {0,25,50,100,250,500} ms (×3 repetitions, `scripts/exp_latency_sweep.sh`), which `analyze.py` renders as a p99-vs-latency curve. Synchronous strategies degrade steeply because every request pays the injected round-trip on its critical path: at 100 ms injected latency the warm p99 rose to ≈844 ms (database), ≈614 ms (Redis), ≈869 ms (payload-hash), and ≈2.0 s (ZooKeeper), and at 500 ms the database and ZooKeeper p99 exceeded 5 s while the asynchronous outbox pipeline saturated the 10 s ceiling. The Kafka strategy was the exception: its API-path p99 stayed at ≈1.8 ms across all injected levels because the side effect—and thus the affected dependency—is off the request path. No duplicate side effects occurred at any latency level (DSEVR = 0 throughout).

Message redelivery under consumer crash (E6). With the Kafka strategy under load we `SIGKILL`ed a consuming service instance mid-processing and restarted it. Its uncommitted offsets were redelivered and reprocessed; the processed-message guard suppressed a mean of **7,572** already-applied operations (6,671–8,482 across three crash-recovery runs) and the duplicate side-effect violation rate remained **0**. Duplicate delivery demonstrably occurred (and was absorbed), confirming H4 under crash-recovery and reinforcing that at-least-once delivery plus an idempotent consumer yields effectively-once *side effects*.

Conflicting payload with the same key (E7). Reusing an idempotency key with a different payload must return `409 Conflict`, not a stale replay. For both the payload-hash and the database strategies, *every* request whose first (establishing) call committed was correctly answered with a conflict on the second call—a conditional detection accuracy of **1.0** (correct = first-applied exactly). A naive key-only design would instead replay the prior result; binding the key to a payload hash is what makes the conflict detectable (H6).

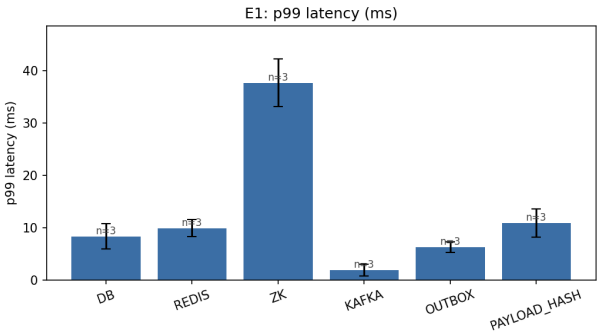

Fig. 2. E1 p99 latency by strategy (mean of three runs, 95% CI). *Measured.*

Correctness under partition (E4). We injected an 8 s partition of each strategy's coordination dependency mid-load. *All three lock/idempotency strategies were fail-closed: the duplicate side-effect violation rate stayed at 0* (Table III). A clean partition makes the store *unavailable*, not unsafe—requests error out rather than producing a second side effect. The strategies differ in availability and recovery, not correctness: the database saw the highest request error rate (14.2%) and slowest API recovery (≈2.45 s) because it is the system of record; Redis recovered fastest (≈0.57 s, 8.6% errors); ZooKeeper had the lowest error rate (2.6%) with a ≈0.91 s recovery as the Curator session re-established (Figs. 4, 5).

Lock timing and clock drift (E5). Skewing one service node's wall clock by +2 s (`libfaketime`) produced *no* premature expirations, stale locks, or violations for either the Redis or the ZooKeeper lock: Redisson enforces its lease via the Redis server's clock (server-side `PEXPIRE`), and ZooKeeper ownership is session/quorum-bound—both are robust to *client* clock drift, unlike a naive client-timed `SET NX PX` lock. We then stressed the timing assumption directly by shortening the Redis lease to 300 ms and injecting 200 ms of Redis latency so the lease can lapse mid critical-section. This produced a mean of **168 premature lease expirations** (163–178 across three runs; Fig. 6)—empirically reproducing the hazard Kleppmann describes [21]—*yet the duplicate side-effect violation rate remained 0*. The lock's timing guarantee broke, but the layered design (lock plus an in-critical-section idempotency-marker recheck, with the side effect itself guarded by the database) still admitted at most one side effect. This is the paper's sharpest finding: *a lock used for mutual exclusion is not by itself a correctness mechanism; binding it to an idempotency record is what prevents duplicate side effects when the lock's assumptions fail.*

TABLE I

STRATEGY COMPARISON MATRIX (QUALITATIVE SUMMARY). EACH COLUMN IS SCOPED TO THE SCENARIO THAT DEFINES IT: THROUGHPUT AND P99 FROM THE E1 BASELINE, DSEVR FROM THE E2 BURST, PARTITION BEHAVIOR FROM E4, AND CLOCK-DRIFT BEHAVIOR FROM E5; CELLS ARE NOT BLENDED ACROSS SCENARIOS. THE THROUGHPUT/P99 CELLS REFLECT THE PILOT E1 RUN AND ARE SUPERSEDED BY THE OPEN-LOOP SATURATION SWEEP FOR ABSOLUTE CEILINGS (SEE TABLE II).

Strategy	Thr.	p99	DSEVR	Partition	Clock-drift
PostgreSQL unique	Med	Med/Hi	≈0	DB-dep.	Low
Redis lock	High	Low	Poss. fault	Sensitive	High
ZooKeeper/etcd	Lo/Med	High	≈0	Stronger	Low
Kafka idempotent	High	Pipe-dep.	Low	Med/Strong	Low
Outbox+consumer	Med	Med	≈0	Stronger e2e	Low
Payload-hash	Med	Med	—	—	—

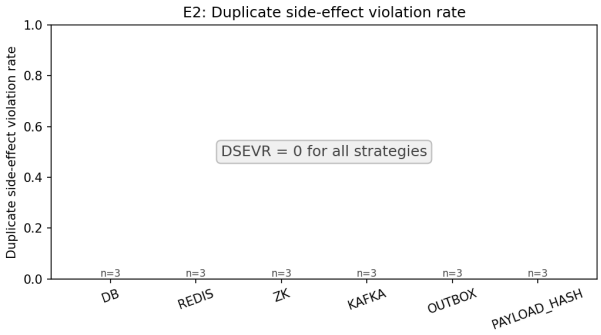

Fig. 3. Duplicate side-effect violation rate under burst (E2): zero for all strategies. *Measured.*

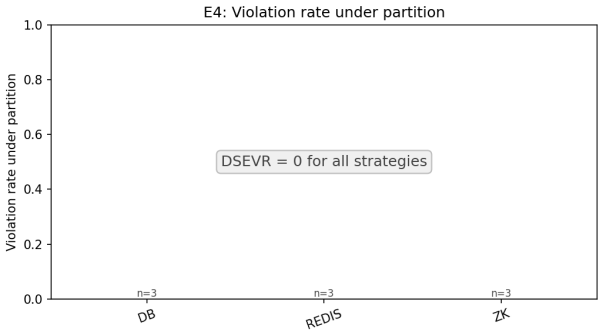

Fig. 4. Duplicate side-effect violation rate under network partition (E4): zero for all strategies (fail-closed). *Measured.*

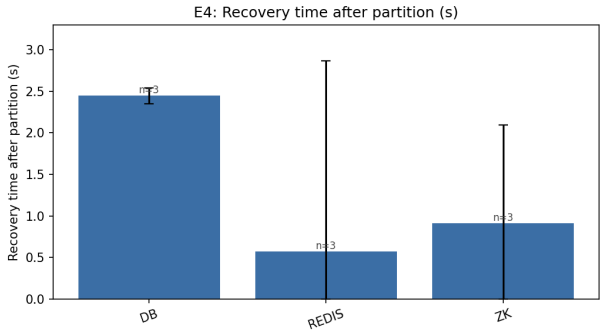

Fig. 5. API recovery time after an 8 s partition (E4). *Measured.*

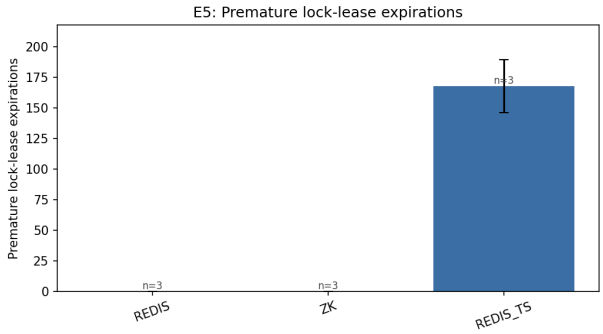

Fig. 6. Premature lock-lease expirations (E5): zero under client clock drift for Redis (server-side TTL) and ZooKeeper (session-based), but a mean of 168 under a short-lease + injected-latency timing stress (REDIS_TS) — which nonetheless produced zero duplicate side effects. *Measured.*

IX. DISCUSSION

Perfectly achieving exactly-once delivery is theoretically impossible in a distributed system. The Two Generals Problem—proven in 1975—shows that reliable agreement cannot be guaranteed over an unreliable channel. The CAP theorem further constrains the space: under partition, a system must choose between consistency and availability. These are not engineering limitations to be overcome; they are mathematical boundaries. The practical question is therefore not whether duplicate delivery will occur—it will—but whether the system can absorb it without producing duplicate side effects. This reframing is the central contribution of this work: idempotency is not about preventing duplicate delivery, it is about making duplicate delivery irrelevant.

The most surprising result was not which strategy performed best, but what happened when the lock's timing assumptions were deliberately violated: 168 premature lease expirations occurred, yet not a single duplicate side effect was produced. This suggests that practitioners often over-rely on distributed locks for correctness when the real guarantee comes from the idempotency record underneath.

The results delineate a performance/correctness frontier rather than a single winner. Low-latency Redis locking is attractive under benign conditions but its correctness leans on timing assumptions that partition and clock drift can violate; consensus coordination buys robustness at a latency/throughput cost; database-level idempotency offers strong correctness with contention-driven tail latency under

TABLE II
MEASURED RESULTS: BASELINE THROUGHPUT AND P99 ON THE WARM RUN (E1; COLD WARM-UP RUN EXCLUDED), AND DUPLICATE SIDE-EFFECT VIOLATION RATE UNDER BURST (E2).

Strategy	Thr. (req/s)	p99 (ms)	DSEVR (E2)
DB	200.0	8.1	0.0000
REDIS	200.0	9.9	0.0000
ZK	199.9	36.9	0.0000
KAFKA	200.0	1.6	0.0000
OUTBOX	200.0	5.9	0.0000
PAYLOAD_HASH	200.0	10.4	0.0000

TABLE III
MEASURED FAULT BEHAVIOUR: E4 REPORTS REQUEST ERROR RATE AND API RECOVERY; E5 REPORTS PREMATURE LEASE EXPIRATIONS. DSEVR (DUPLICATE SIDE-EFFECT VIOLATION RATE) IS 0 THROUGHOUT.

Scenario	Strategy	Error / Premature	Recovery	DSEVR
E4 partition	DB	14.2%	2.45 s	0
E4 partition	REDIS	8.6%	0.57 s	0
E4 partition	ZK	2.6%	0.91 s	0
E5 timing	REDIS	0 prem.	–	0
E5 timing	ZK	0 prem.	–	0
E5 timing	REDIS_TS	168 prem.	–	0

bursts; event-driven idempotency cannot prevent duplicate *delivery* but, with a processed-message store, prevents duplicate *side effects*. This last point motivates careful use of "exactly-once" terminology: we distinguish exactly-once *delivery* (generally unattainable), exactly-once *processing*, and exactly-once *side effects* ("effectively-once"), the last being the achievable and meaningful guarantee.

X. DESIGN GUIDELINES

G1: Use database-level idempotency as a strong baseline when side effects are primarily database-bound and correctness outranks ultra-low latency. **G2**: Use Redis locks only when lock duration, TTL, clock behavior, and network assumptions are understood and monitored; add fencing tokens for correctness-critical sections. **G3**: Prefer consensus-based coordination when correctness under partition outranks latency. **G4**: Make consumers idempotent in event-driven systems because duplicate delivery cannot be eliminated. **G5**: Bind idempotency keys to payload hashes to detect semantic conflicts. **G6**: Measure correctness metrics, not only throughput and latency—three of the eight engineering issues found during this study were invisible to throughput measurement and surfaced only through the out-of-band correctness audit. **G7**: Use fault injection before production to validate idempotency assumptions. **G8**: Use the transactional outbox when business transactions and event publication must be coordinated reliably.

XI. THREATS TO VALIDITY

Synthetic workloads may not capture all real traffic; JVM GC, broker/store configuration, and hardware limits influence absolute numbers; the chosen fault set does not cover all failures; and only six strategies are evaluated.

Single-host bias (directional). All inter-component paths in our deployment are loopback; the Toxiproxy layer injects *artificial* latency but the underlying transport is local. Real coordination over a LAN adds physical round-trips that our setup omits, so our coordination-overhead figures are *lower bounds*. The bias is strategy-dependent and predictable in sign and rough magnitude. ZooKeeper's `InterProcessMutex` acquire performs at least one client→leader round-trip plus quorum replication; over a LAN (∼0.2–1 ms per RTT, several RTTs per acquire) this adds on the order of 1–5 ms per acquired lock, so the measured 37.7 ms p99 understates production coordination cost by roughly that margin. A Redis lock acquire/release is a single client→server round-trip, adding on the order of 0.5–2 ms over a LAN relative to loopback. The database and Kafka paths are similarly understated by one network RTT per synchronous interaction. These are directional estimates, not measurements; a multi-node replication of the testbed is the way to calibrate them, and we leave it to future work.

Statistical scale. The performance numbers are pilot-scale (see the replication protocol above); their magnitudes carry wide uncertainty and the throughput convergence is a load-path artifact rather than a finding. The *correctness* conclusions (DSEVR $= 0$) are categorical and do not depend on replication count.

We mitigate the remaining threats by pinning component versions, verifying correctness from an out-of-band audit table independent of in-app counters, and releasing the full artifact—including the open-loop saturation and latency-sweep campaigns—for independent re-measurement.

XII. REPRODUCIBILITY AND ARTIFACT AVAILABILITY

The source code, Docker-based testbed, workload and fault scripts, monitoring configuration, raw results, and analysis scripts are released publicly; a versioned release is archived on Zenodo for a persistent DOI. The artifact repository is publicly available at https://github.com/mahmutkarabulut1/idempotency-strategies-evaluation. A single command brings up the stack; the experiment runner executes the full grid; analysis scripts regenerate every figure and table. All workloads are synthetic and contain no real user, payment, or personal data.

ACKNOWLEDGMENTS

The author used AI-assisted tools (Claude, Anthropic) for implementation support, including test harness development, shell scripting, and analysis pipeline construction. The research design, experimental methodology, fault injection scenarios, result interpretation, and all conclusions are solely the author's own work.

XIII. CONCLUSION

The central takeaway is that idempotency is not a single mechanism but a layered property: a lock provides mutual exclusion; a database constraint provides correctness; together they provide both. Practitioners who deploy only one layer are making an assumption about the other—an assumption that

partition, clock drift, or a GC pause can silently invalidate. Distributed systems cannot be designed against network unreliability; they must be designed with it. Idempotency is the engineering response to this irreducible uncertainty, and measuring it requires correctness metrics that throughput and latency alone cannot reveal. We built a reproducible, fault-injected testbed and compared six idempotency and locking strategies under identical workloads, evaluating both performance and a domain-independent correctness metric. The study maps the trade-offs among low-latency locking, database-enforced correctness, consensus coordination, and event-driven deduplication, and offers actionable guidelines. Future work will extend this comparison to multi-node deployments, explore etcd as an alternative coordination layer, and investigate fencing token integration to harden Redis-based strategies against the timing hazards demonstrated in E5.

REFERENCES

[1] Stripe, "Designing robust and predictable apis with idempotency," https://stripe.com/blog/idempotency, 2017, accessed 2026.

[2] J. Chew, "Avoiding double payments in a distributed payments system," https://medium.com/airbnb-engineering, 2018, airbnb Tech Blog. Accessed 2026.

[3] B. Leels, "Implementing stripe-like idempotency keys in Postgres," https://brandur.org/idempotency-keys, 2017, accessed 2026.

[4] M. Burrows, "The Chubby lock service for loosely-coupled distributed systems," in *Proc. 7th USENIX Symp. Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2006, pp. 335–350.

[5] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, "ZooKeeper: Wait-free coordination for internet-scale systems," in *Proc. USENIX Annual Technical Conference (ATC)*. USENIX Association, 2010.

[6] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annual Technical Conference (ATC)*. USENIX Association, 2014, pp. 305–319.

[7] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017.

[8] M. Whitaker, B. Volckaert, and M. Al-Naday, "Idempotency in service mesh: For resiliency of fog-native applications in multi-domain edge-to-cloud ecosystems," in *Proc. 15th Int. Conf. Cloud Computing and Services Science (CLOSER)*. SciTePress, 2025, pp. 182–189.

[9] D. da Palma Pereira and A. Rito Silva, "A domain-driven design simulator for business logic-rich microservice systems," https://arxiv.org/abs/2605.01159, 2026.

[10] Uplatz, "A systematic analysis of distributed locking: A comparative study of Redis, Zookeeper, and consensus-based coordination," https://uplatz.com/blog, 2025, industry blog. Accessed 2026.

[11] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[12] J. C. Corbett, J. Dean *et al.*, "Spanner: Google's globally-distributed database," in *Proc. 10th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2012, pp. 251–264.

[13] P. Helland, "Life beyond distributed transactions: An apostate's opinion," in *Proc. 3rd Biennial Conf. Innovative Data Systems Research (CIDR)*, 2007, pp. 132–141.

[14] H. Garcia-Molina and K. Salem, "Sagas," *ACM SIGMOD Record*, vol. 16, no. 3, pp. 249–259, 1987.

[15] C. Richardson, "Microservices pattern: Saga," https://microservices.io/patterns/data/saga.html, accessed 2026.

[16] E. Daraghmi, C.-P. Zhang, and S.-M. Yuan, "Enhancing saga pattern for distributed transactions within a microservices architecture," *Applied Sciences*, vol. 12, no. 12, p. 6242, 2022.

[17] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.

[18] C. Richardson, "Microservices pattern: Transactional outbox," https://microservices.io/patterns/data/transactional-outbox.html, accessed 2026.

[19] G. DeCandia, D. Hastorun *et al.*, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP)*, 2007, pp. 205–220.

[20] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.

[21] M. Kleppmann, "How to do distributed locking," https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html, 2016, accessed 2026.

[22] S. Sanfilippo, "Is Redlock safe?" http://antirez.com/news/101, 2016, accessed 2026.

Mahmut Karabulut is a final-year Computer Engineering student at Ege University, Izmir, Turkey. His research interests include distributed systems reliability, idempotency, microservices architecture, and fault-injected experimental evaluation.